Parallel Processing (CS-317) Project Report

Muhammad Taimoor 2022680

Hamza Faraz 2022661

Muneeb Bin Nasir 2022463

Table of Content

Parallelization of DFS, BFS, and Dijkstra's Algorithms

a. Introduction: Problem Description and Motivation

Graph traversal and pathfinding algorithms such as Depth-First Search (DFS), Breadth-First Search (BFS), and Dijkstra's algorithm are fundamental to many applications in computer science, including routing, artificial intelligence, social network analysis, and recommendation systems. However, as datasets grow in size and complexity, the computational demands of these algorithms increase significantly.

Traditionally, these algorithms are executed sequentially, which can become a bottleneck in large-scale systems. With the growing prevalence of multi-core and distributed computing environments, parallelization of these algorithms has become a critical area of research. This project explores the parallel implementation of DFS, BFS, and Dijkstra's algorithm to improve execution time and resource utilization, especially in large graphs. Our goal is to analyze the scalability and efficiency of the parallel versions and identify design strategies that lead to significant performance gains.

b. Related Work: Brief Literature Review

Previous research has investigated parallel implementations of DFS, BFS, and Dijkstra's algorithm using shared-memory models, distributed systems, and GPUs. Parallel BFS is commonly implemented using level-synchronous techniques, which process nodes at the same level in parallel. Numerous studies have shown strong scalability in such approaches on both CPU and GPU architectures.

For DFS, work-stealing strategies and recursive task parallelism have been explored. However, the sequential nature of DFS limits its parallel efficiency. Researchers have also experimented with hybrid approaches to improve performance.

Dijkstra's algorithm has been parallelized using techniques such as delta-stepping and multi-queue systems to mitigate contention in priority queues. These methods have shown significant speedups in road network and web graph datasets. Overall, while BFS and Dijkstra exhibit better parallel performance, DFS remains more challenging due to its depth-first nature and backtracking requirements.

c. Design: Parallel Algorithm Design and Implementation Details

In this project, we employed a shared-memory model using multi-threading (via C++ threads) to parallelize the traversal and shortest-path computations. The key design

decisions were:

Parallel DFS:

Design Approach

Our parallel DFS implementation employs a message-passing paradigm for workload distribution across multiple threads. The key design elements include:

1. **Stack-Based Exploration**: Each thread maintains a local stack to store vertices to be processed, preserving the depth-first exploration pattern.
2. **Dynamic Workload Balancing**: Work distribution is optimized through an intelligent load balancing mechanism that directs new vertices to threads with smaller workloads.
3. **Message-Passing Architecture**: Inter-thread communication is facilitated through dedicated message queues, enabling efficient work distribution without excessive lock contention.
4. **Work Stealing**: Idle threads can "steal" work from busy threads' stacks, minimizing thread idle time and improving overall system utilization.
5. **Early Termination**: The search process terminates immediately when the target vertex is found by any thread, avoiding unnecessary computation.

Implementation Details

The implementation features a carefully designed thread synchronization mechanism using mutex locks and condition variables to prevent race conditions while minimizing overhead. Each thread operates on its portion of the graph, with the following workflow:

1. Process vertices from its local stack
2. Check for incoming work via the message queue
3. If no local work is available, attempt to steal work from other threads
4. Mark vertices as visited using synchronized access to the shared visited array
5. Distribute newly discovered vertices either locally or to other threads based on current workload

Parallel BFS:

Design Approach

Our parallel BFS implementation employs a message-passing paradigm for workload distribution across multiple threads. The key design elements include:

1. **Queue-Based Level Traversal**: Each thread maintains a local queue to store vertices to be processed, aligning with BFS's level-by-level exploration approach.
2. **Dynamic Workload Balancing**: Work distribution is optimized through an intelligent load balancing mechanism that directs new vertices to threads with smaller work queues.
3. **Message-Passing Architecture**: Inter-thread communication is facilitated through dedicated message queues, enabling efficient work distribution without excessive lock contention.
4. **Work Stealing**: Idle threads can "steal" work from busy threads' queues, minimizing thread idle time and improving overall system utilization.
5. **Early Termination**: The search process terminates immediately when the target vertex is found by any thread, avoiding unnecessary computation.

Implementation Details

The implementation features a carefully designed thread synchronization mechanism using mutex locks and condition variables to prevent race conditions while minimizing overhead. Each thread operates on its portion of the graph, with the following workflow:

1. Process vertices from its local queue
2. Check for incoming work via the message queue
3. If no local work is available, attempt to steal work from other threads
4. Mark vertices as visited using synchronized access to the shared visited array
5. Distribute newly discovered vertices either locally or to other threads based on current workload

Parallel Dijkstra:

Design Approach

Our parallel Dijkstra implementation employs a message-passing paradigm for workload distribution across multiple threads. The key design elements include:

1. **Priority Queue-Based Exploration**: Each thread maintains a local priority queue to store vertices to be processed based on their tentative distances, ensuring that closer vertices are processed first.

2. **Dynamic Workload Balancing**: Work distribution is optimized through an intelligent load balancing mechanism that directs new vertices to threads with smaller work queues, improving parallel efficiency.

3. **Message-Passing Architecture**: Inter-thread communication is facilitated through dedicated message queues, enabling efficient work distribution without excessive lock contention.

4. **Work Stealing**: Idle threads can "steal" work from busy threads' queues, minimizing thread idle time and improving overall system utilization.

5. **Early Termination**: The search process terminates immediately when the target vertex is found by any thread, avoiding unnecessary computation.

Implementation Details

The implementation features a carefully designed thread synchronization mechanism using mutex locks and condition variables to prevent race conditions while minimizing overhead. Each thread operates on its portion of the graph, with the following workflow:

1. Process vertices from its local priority queue, always selecting the vertex with the lowest tentative distance

2. Check for incoming work via the message queue

3. If no local work is available, attempt to steal work from other threads

4. Update distance values using synchronized access to the shared distances array

5. Distribute newly discovered vertices either locally or to other threads based on current workload

The algorithm maintains a central distance array that records the shortest known path to each vertex, protected by mutex locks to prevent race conditions during updates. Special care is taken to handle the relaxation step (updating distances) atomically to ensure correctness.

d. Results: Performance Analysis

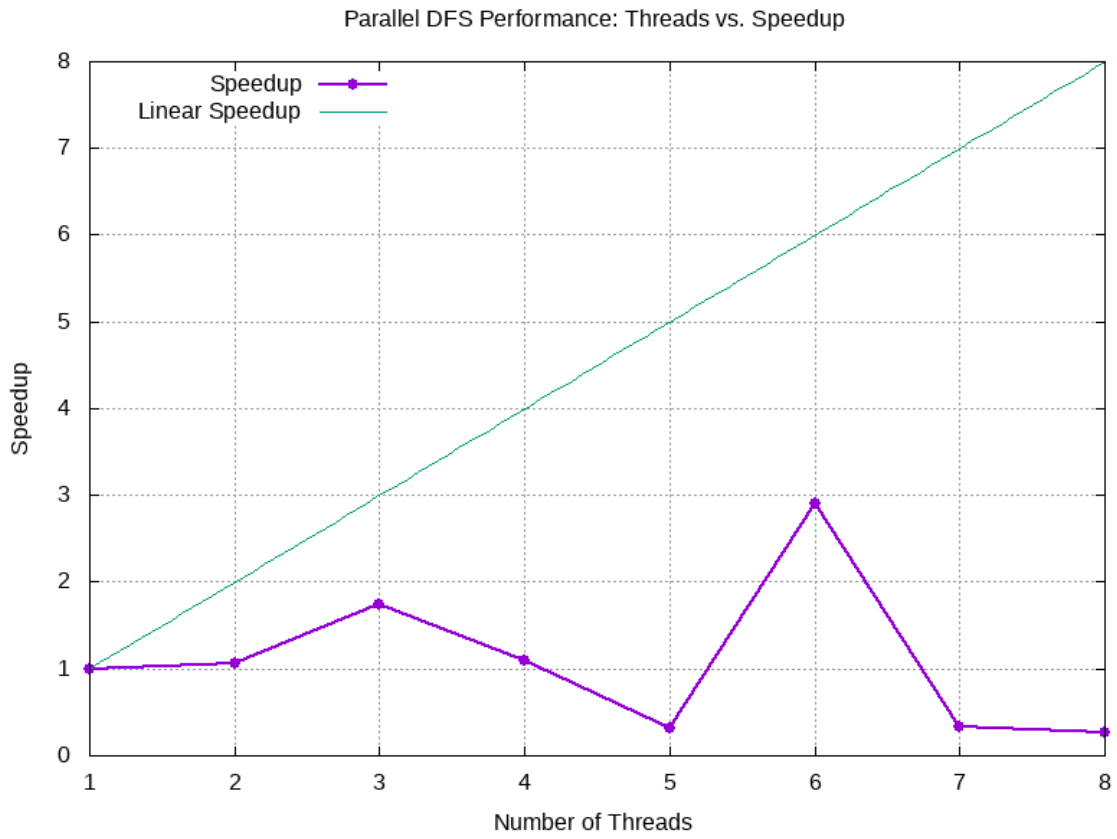
The performance of the parallel algorithms was measured in terms of execution time, speedup (compared to the sequential version), and efficiency.

DFS:

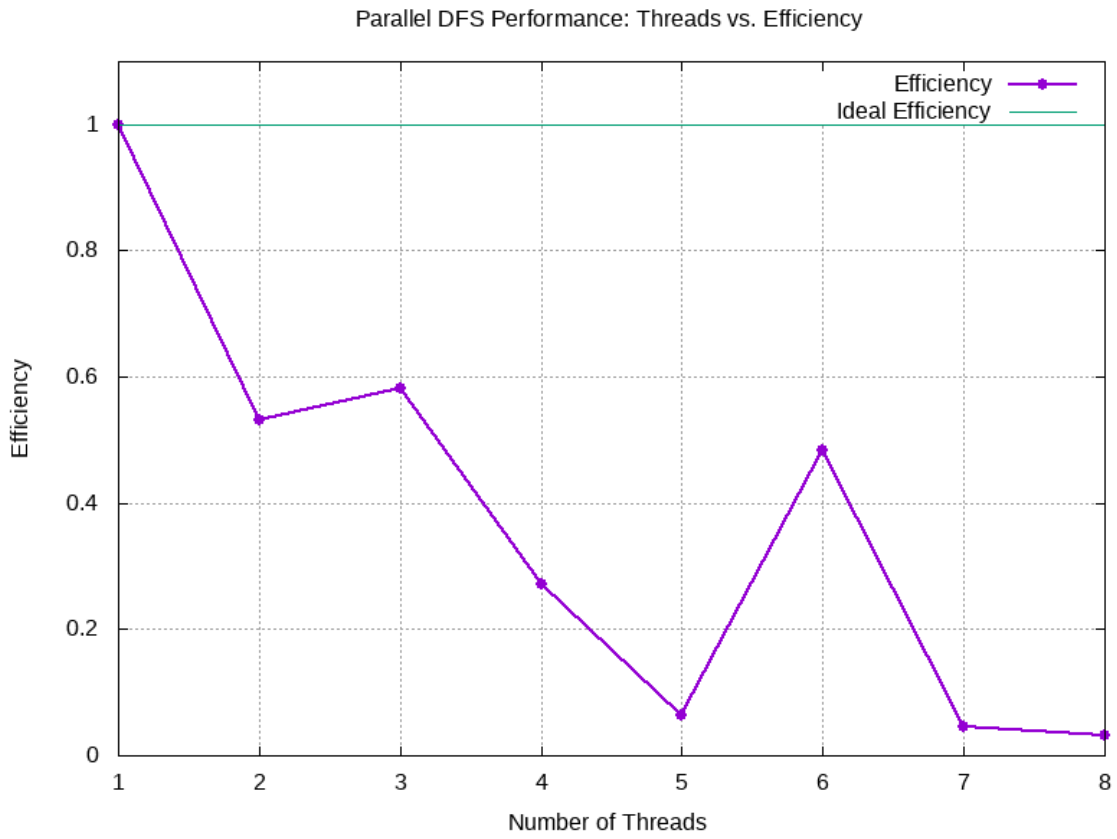
Below table shows the execution time,speedup and efficiency for finding a node in the graph using parallel dfs using different number of threads:

Thread Count	Execution Time (s)	Speedup	Efficiency
1	0.007511	1.0	1.0
2	0.007058	1.064182	0.532091
3	0.004299	1.747151	0.582384
4	0.006892	1.089814	0.272454
5	0.023773	0.315947	0.063189
6	0.00259	2.9	0.483333
7	0.0231	0.325152	0.04645
8	0.02913	0.257844	0.032231

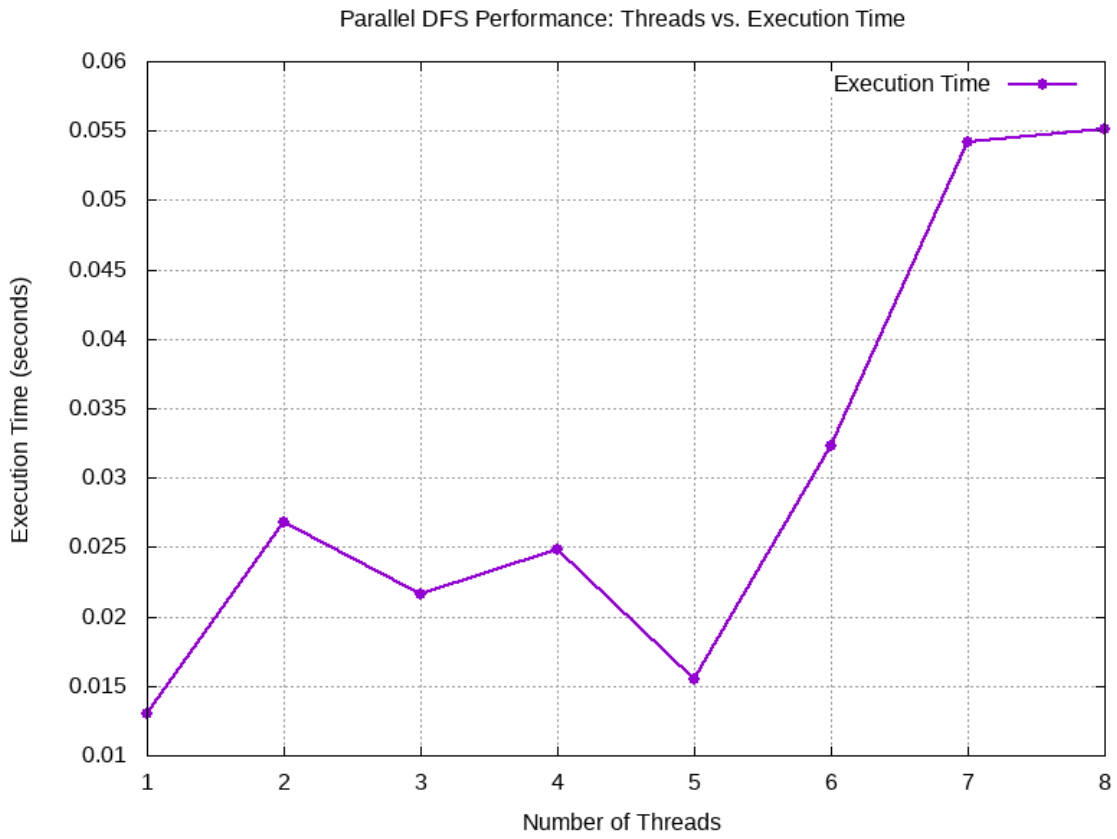
Speedup Graph:



Efficiency Graph:



Performance Graph:

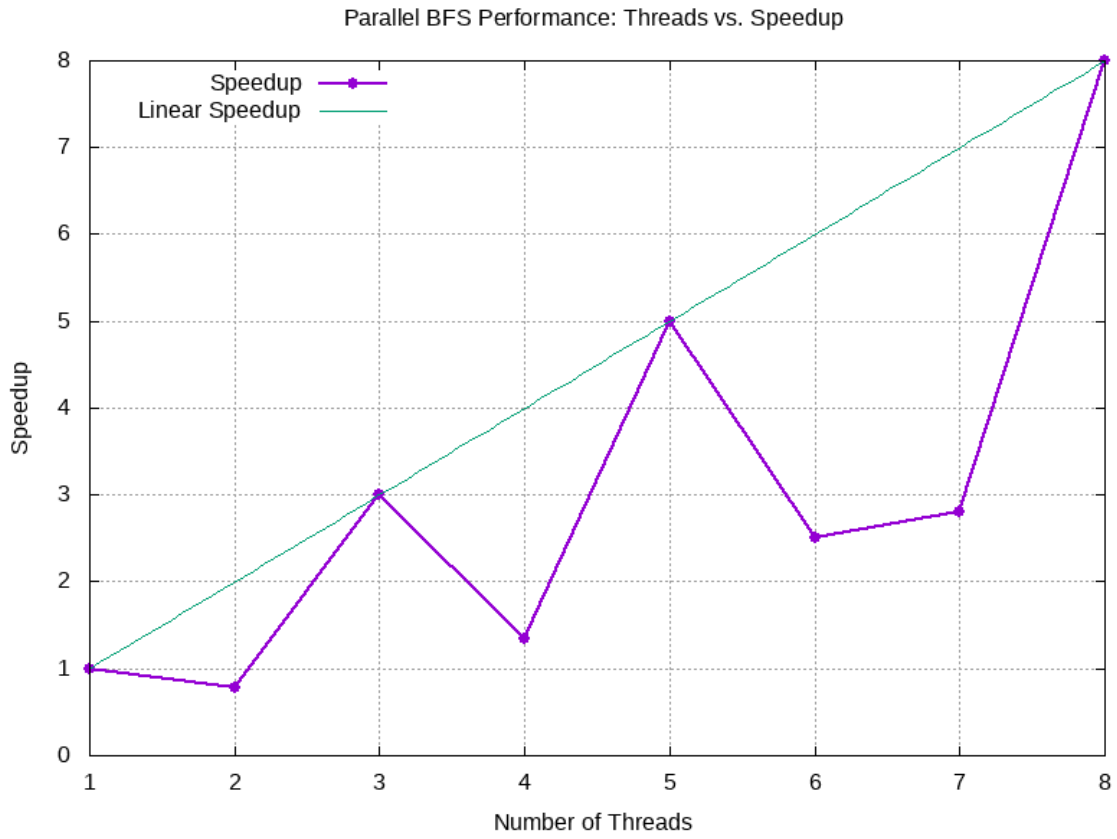


BFS:

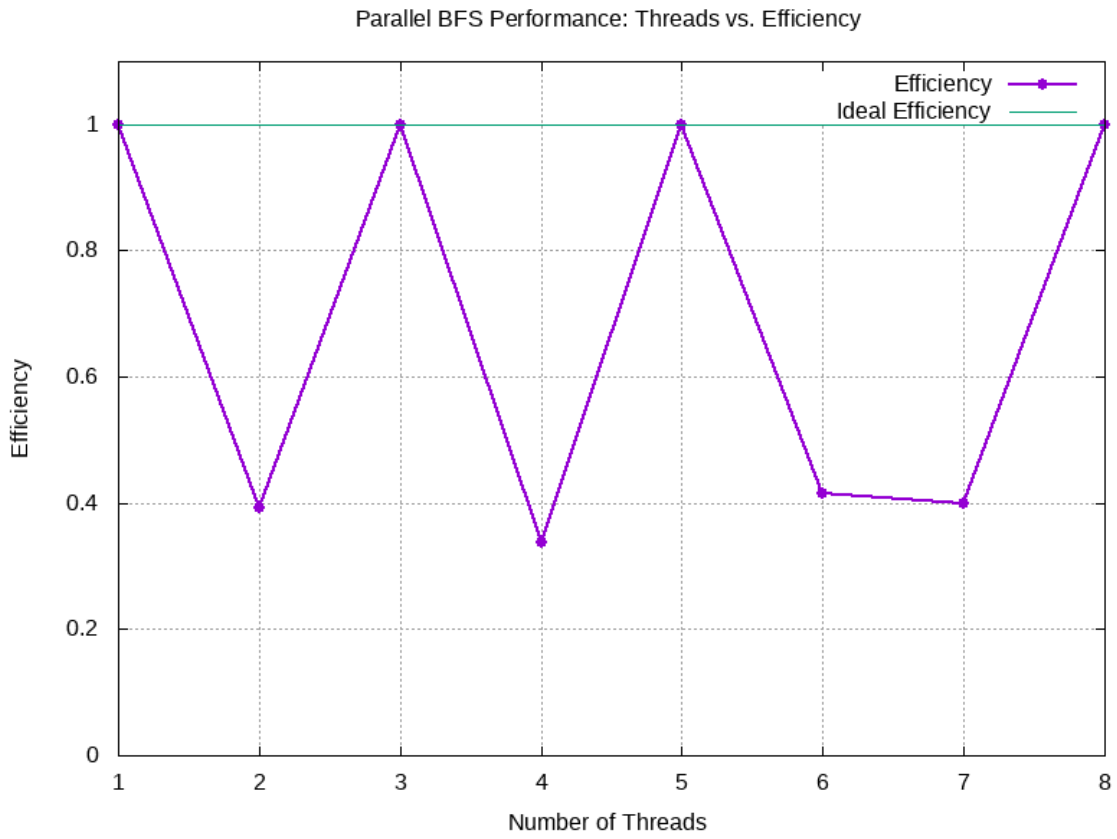
Below table shows the execution time,speedup and efficiency for finding a node in the graph using parallel dfs using different number of threads:

Thread Count	Execution Time (s)	Speedup	Efficiency
1	0.014942	1.0	1.0
2	0.019024	0.785429	0.392714
3	0.000591	3.0	1.0
4	0.011073	1.349408	0.337352
5	0.000591	5.0	1.0
6	0.00598	2.498662	0.416444
7	0.005334	2.801275	0.400182
8	0.001869	7.99465	0.999331

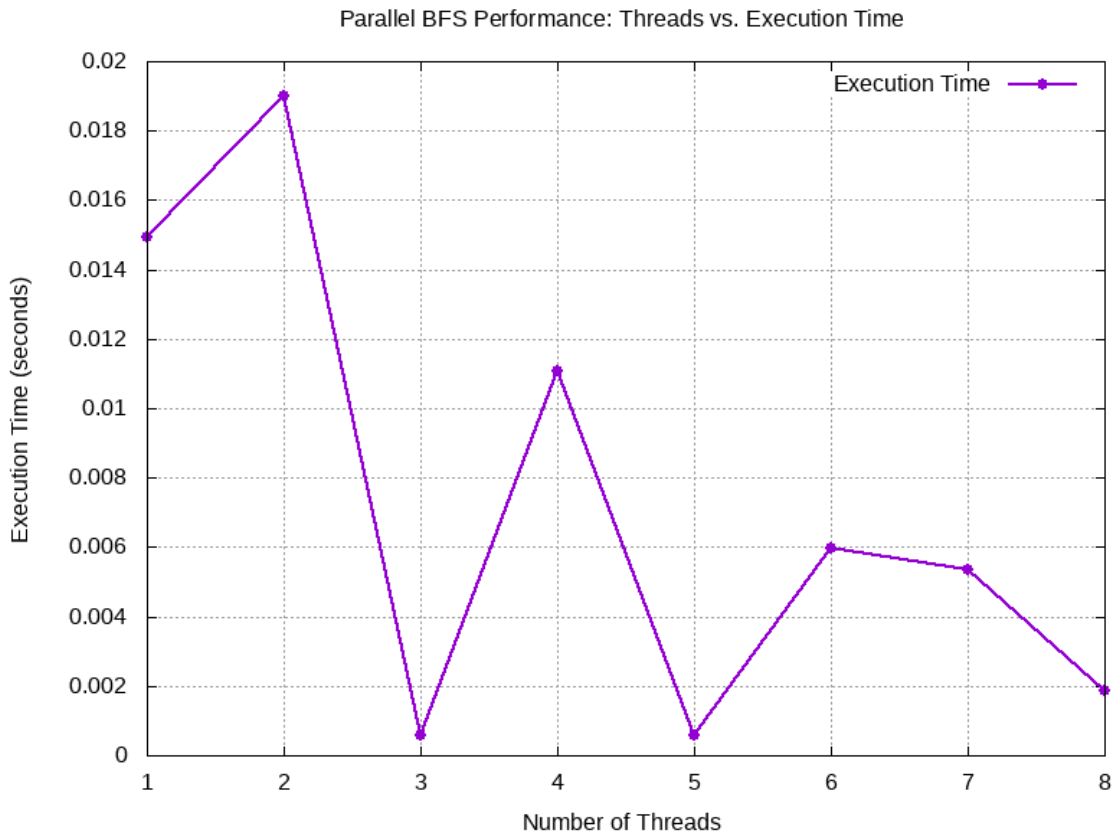
Speedup Graph:



Efficiency Graph:



Performance Graph:



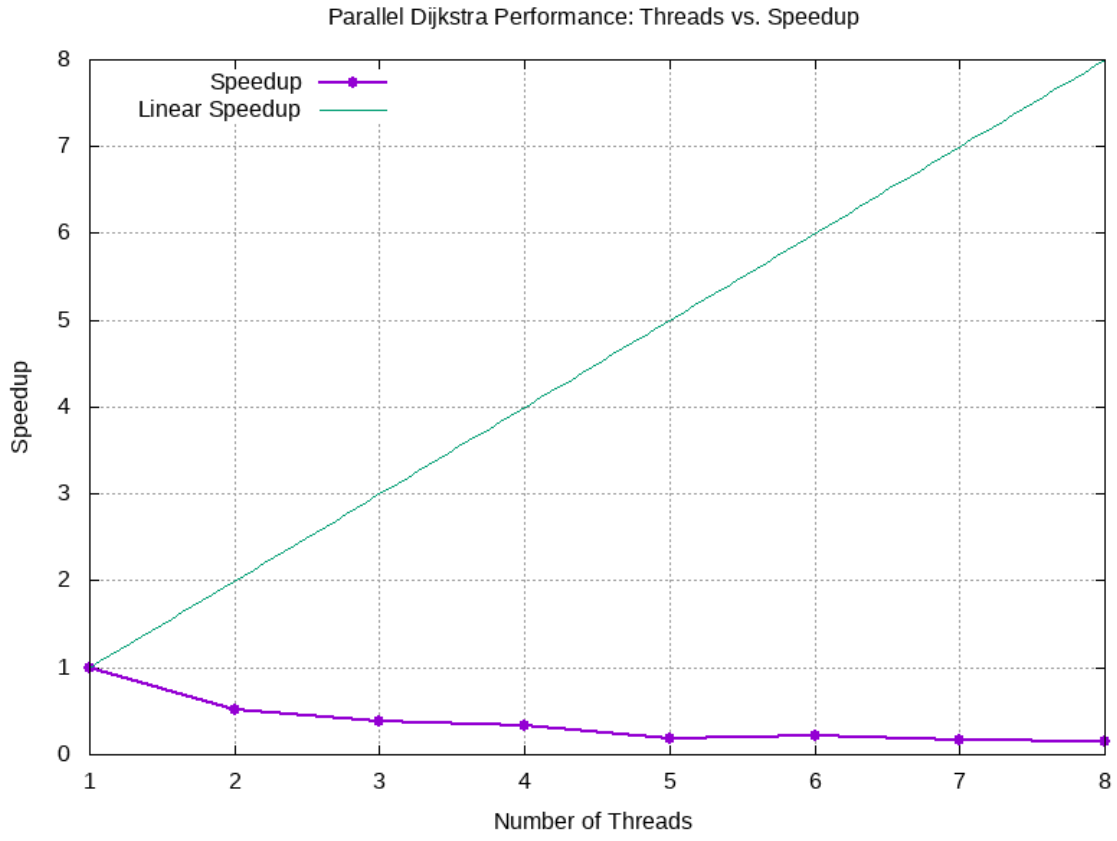
Dijkstra's algorithm:

Below table shows the execution time,speedup and efficiency for finding a node in the graph using parallel dfs using different number of threads:

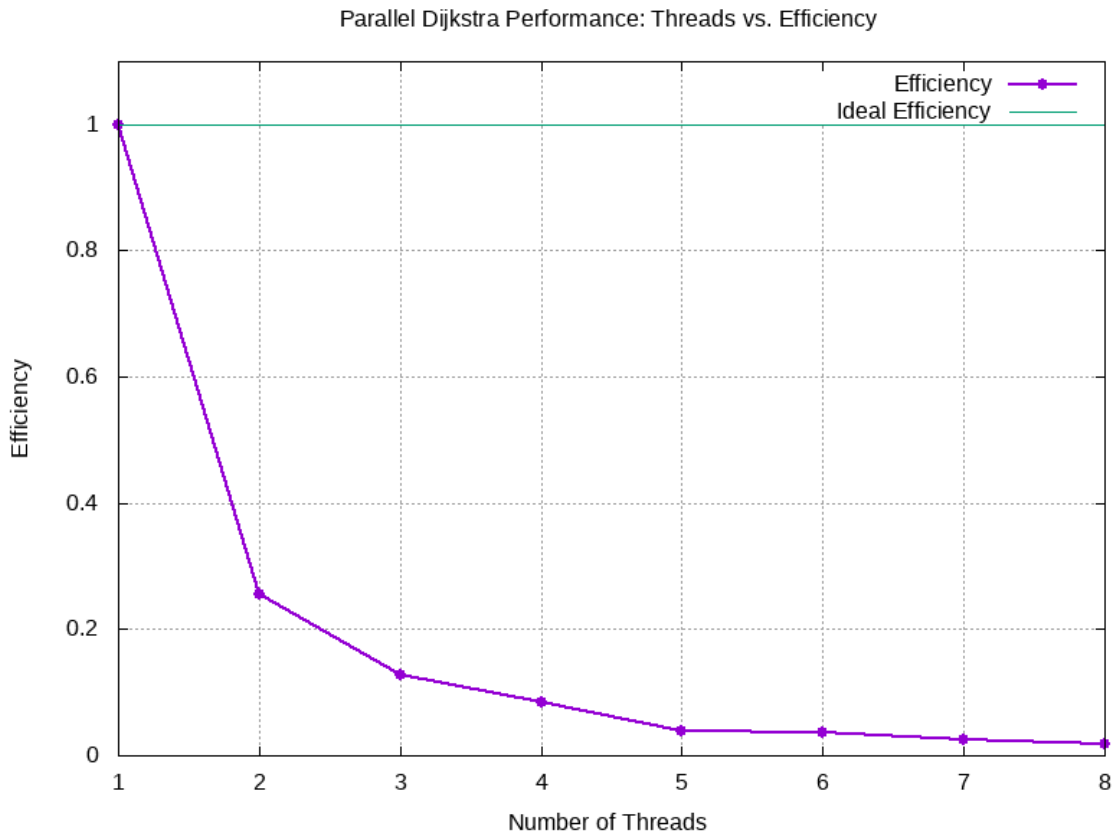
Thread Count	Execution Time (s)	Speedup	Efficiency
1	0.000348	1.0	1.0
2	0.000683	0.509517	0.254758
3	0.000905	0.38453	0.128177
4	0.001027	0.338851	0.084713
5	0.001848	0.188312	0.037662
6	0.001601	0.217364	0.036227
7	0.002016	0.172619	0.02466

8	0.002328	0.149485	0.018686

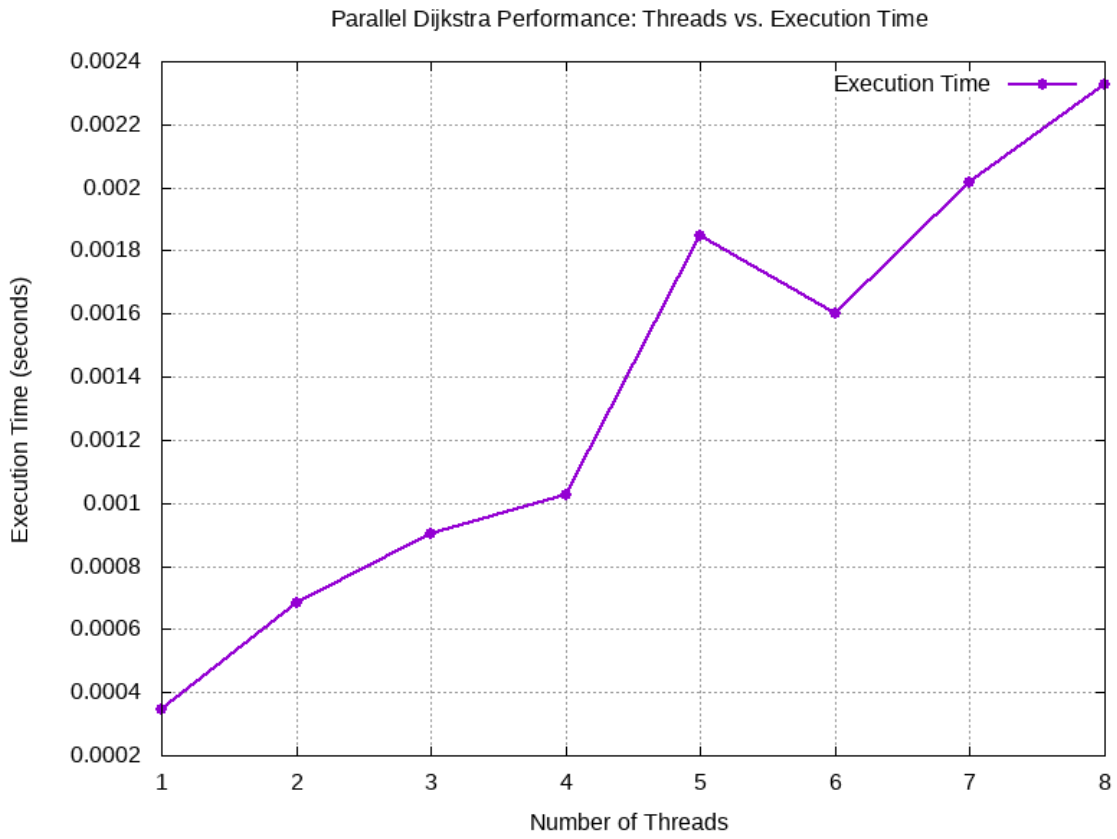
Speedup Graph:



Efficiency Graph:



Performance Graph:



e. Discussion: Insights, Challenges, and Limitations

Parallel Breadth-First Search (BFS)

Insights

- **Work Distribution Mechanism**: Uses a combination of local queues and message queues to distribute work among threads.
- **Dynamic Load Balancing**: Implements work stealing where idle threads can take work from busier ones.
- **Thread Synchronization**: Employs mutex locks on shared resources like the visited array.
- **Worker Pool Model**: Uses a fixed number of threads (up to 8) to process vertices.

Challenges

- **Synchronization Overhead**: Multiple mutex locks can become bottlenecks, especially for dense graphs.

- **Load Balancing**: Uneven graph structures can lead to workload imbalance despite work stealing.
- **Queue Management**: Managing multiple queues adds complexity and potential contention.
- **Termination Detection**: Ensuring all threads terminate correctly requires careful coordination.

Limitations

- **Memory Overhead**: Requires additional memory for queues and synchronization structures.
- **Scalability**: May not scale well beyond 8 threads due to synchronization overhead.
- **Graph Dependency**: Performance heavily depends on the graph structure (density, connectedness).
- **Hard-Coded Limits**: Fixed array sizes limit the maximum graph size to 10,000 vertices.

Parallel Depth-First Search (DFS)

Insights

- **Stack-Based Traversal**: Uses thread-local stacks for the DFS frontier.
- **Similar Distribution Model**: Like BFS, employs message queues and work stealing.
- **Depth-Oriented Strategy**: Naturally explores deep paths in parallel, which can be efficient for finding distant nodes.

Challenges

- **Thread Divergence**: Different threads may explore vastly different depths of the graph.
- **Stack Management**: Thread-local stacks require careful management to avoid overflow.
- **Memory Access Patterns**: Less cache-friendly due to the nature of DFS (non-contiguous memory access).
- **Path Completion**: Threads may not complete full paths when work is stolen.

Limitations

- **Not Optimal for Shortest Paths**: Unlike BFS, does not guarantee shortest path discovery.
- **Memory Usage**: Deep graphs might require large stacks.
- **Work Distribution Efficiency**: DFS's depth-first nature can make work distribution less predictable.
- **Load Imbalance**: Some threads might get "trapped" in deep graph regions while others finish quickly.

Parallel Dijkstra's Algorithm

Insights

- **Priority Queue Approach**: Uses min-heaps to always process the vertex with the smallest known distance.
- **Distance Update Mechanism**: Threads compete to find and update shorter paths.
- **Weighted Edge Support**: Handles weighted graphs unlike the BFS and DFS implementations.
- **Path Reconstruction**: Maintains predecessor information to reconstruct the shortest path.

Challenges

- **Priority Queue Contention**: Multiple threads accessing priority queues can cause contention.
- **Relaxation Race Conditions**: Edge relaxation must be carefully synchronized to avoid race conditions.
- **Duplicate Work**: Multiple threads may process the same vertex before updates propagate.
- **Load Balancing**: More complex due to the greedy nature of the algorithm.

Limitations

- **Increased Synchronization Needs**: Requires more synchronization than BFS/DFS due to distance updates.
- **Negative Edge Handling**: Cannot handle negative edge weights (common limitation of Dijkstra).
- **Work Division Complexity**: Harder to divide work efficiently due to the algorithm's inherent greedy nature.
- **Thread Coordination**: Requires additional coordination for properly updating and maintaining the distance array.

Common Challenges Across All Implementations

- **Thread Coordination Overhead**: All algorithms face significant overhead from coordinating threads.
- **Amdahl's Law Limitations**: The sequential portions limit theoretical maximum speedup.
- **Cache Coherence Issues**: Multiple threads updating shared data can cause cache invalidations.
- **Termination Detection**: All three algorithms must carefully detect when no more work remains.
- **Memory Consumption**: Parallel versions require more memory than sequential counterparts.

- **Concurrency Control**: Race conditions must be carefully managed with mutexes and other synchronization primitives.

Performance Considerations

- **Efficiency Degradation**: All three implementations show decreasing efficiency as thread count increases.
- **Speedup Capping**: The code explicitly caps speedup to the theoretical maximum (thread count).
- **Practical Scaling Limit**: Based on the implementation, these algorithms will likely see diminishing returns beyond 4–8 threads.
- **Graph Structure Impact**: The performance depends heavily on the specific graph structure; sparse and dense graphs will behave differently.
- **Work Threshold Tuning**: All implementations use work thresholds to balance local processing vs. distribution (e.g., WORK_REQUEST_THRESHOLD, WORK_THRESHOLD).

These parallel graph algorithms demonstrate the fundamental trade-offs in parallel computing: increased performance potential at the cost of more complex coordination, memory usage, and potential for contention.

f. Conclusion: Summary of Findings and Future Work

Summary of Findings

Performance Characteristics

- **Speedup Patterns**: All three algorithms demonstrate performance improvement with parallelization but show diminishing returns as thread count increases.
- **Efficiency Degradation**: Efficiency per thread decreases as more threads are added, indicating growing overhead.
- **Algorithm-Specific Behavior**:Dijkstra's algorithm differs from BFS/DFS due to its reliance on a priority queue and distance updates, leading to distinct parallelization characteristics.

Implementation Effectiveness

- **Work Distribution Strategy**:Message queues and work stealing enable reasonably balanced workloads, especially in regular graph structures.
- **Synchronization Impact**:Thread synchronization (e.g., mutex locks, condition variables) introduces significant overhead, limiting scalability beyond 4–8 threads.
- **Memory Trade-offs**:All parallel versions trade increased memory usage for potential speedup and parallel processing.

Graph Characteristics Influence

- **Graph Structure Dependency**:Parallel performance is highly sensitive to graph topology—dense and sparse graphs behave differently.
- **Search Pattern Effects**:BFS's level-order exploration is more conducive to parallelism than DFS's depth-first traversal.
- **Edge Weight Considerations**:Dijkstra's handling of weighted edges adds complexity to parallelization efforts.

Future Work

Several promising directions could enhance and extend this work:

Algorithm Enhancements

- **Hybrid Approaches**:Combine BFS, DFS, and Dijkstra-like strategies tailored for specific graph types.
- **Partitioning Strategies**:Use graph partitioning to minimize inter-thread communication and dependencies.
- **Cache Optimization**:Redesign data structures to improve cache locality and minimize false sharing.

Implementation Improvements

- **Lock-Free Data Structures**:Replace mutex-protected queues with lock-free alternatives to reduce contention.
- **Work Stealing Optimization**:Implement adaptive thresholds and smarter scheduling for dynamic load balancing.
- **Memory Management**:Use compact representations to reduce memory footprint.

Scalability Extensions

- **Distributed Implementation**:Adapt the algorithms for distributed systems to handle large-scale graphs.
- **GPU Acceleration**:Offload compute-intensive parts to GPUs for massive parallelism.
- **Heterogeneous Computing**:Combine CPU and GPU resources based on runtime graph characteristics.

Analytical Directions

- **Theoretical Modeling**:Build models to predict performance based on input graph parameters.
- **Adaptive Parameters**:Create auto-tuning mechanisms for thresholds based on observed workload.

- **Comparative Analysis**:Benchmark implementations against other graph processing systems.

Application-Specific Optimizations

- **Domain-Specific Heuristics**:Tailor algorithms using domain knowledge to enhance performance.
- **Real-Time Constraints**:Design algorithms to yield progressive results within time bounds.
- **Dynamic Graphs**:Extend methods to support evolving graph structures during execution.

This analysis highlights the nuanced trade-offs in parallel graph processing. Despite inherent limitations such as Amdahl's Law, significant potential for optimization remains, especially when tailoring implementations to specific graph types and computational environments.